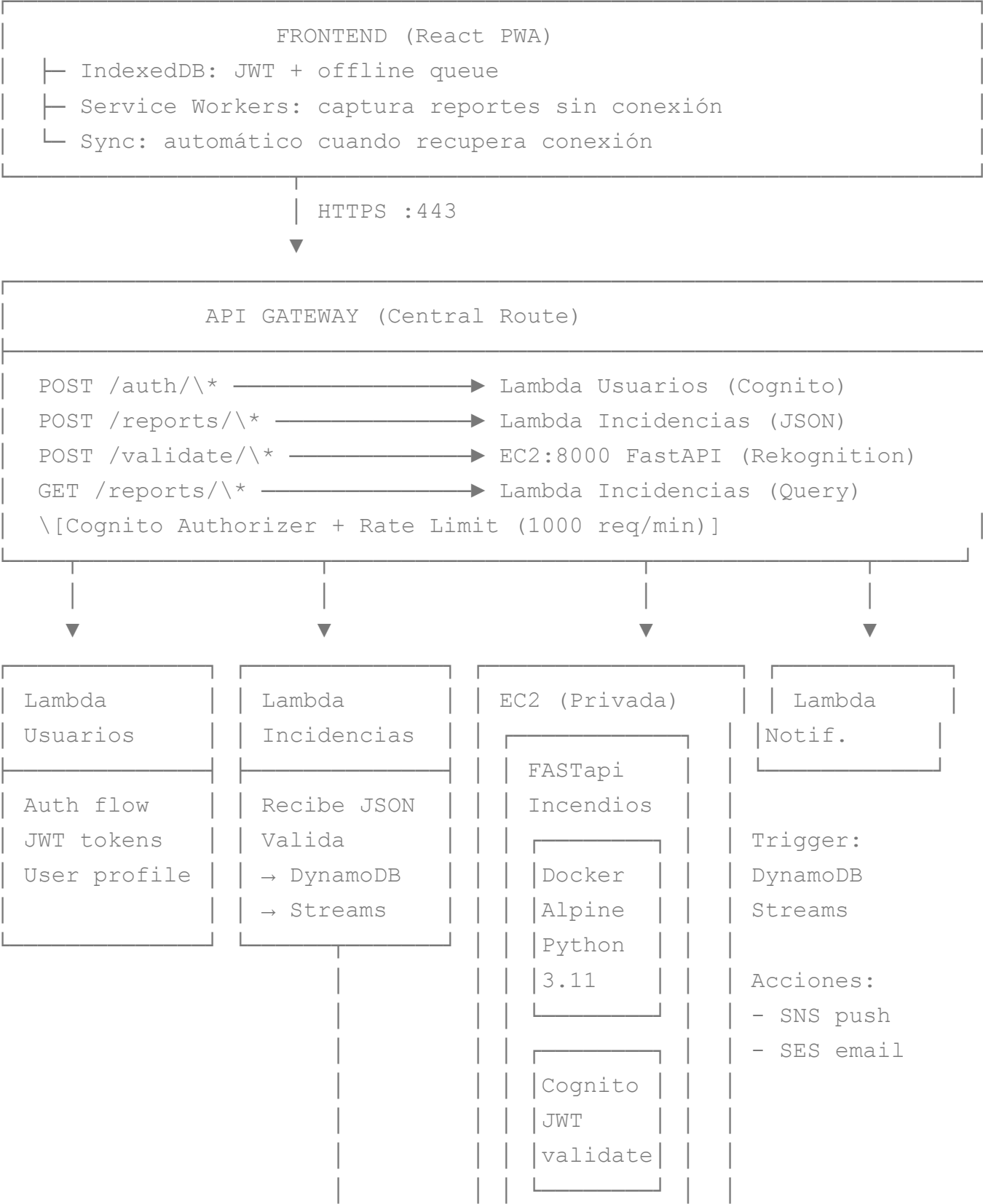
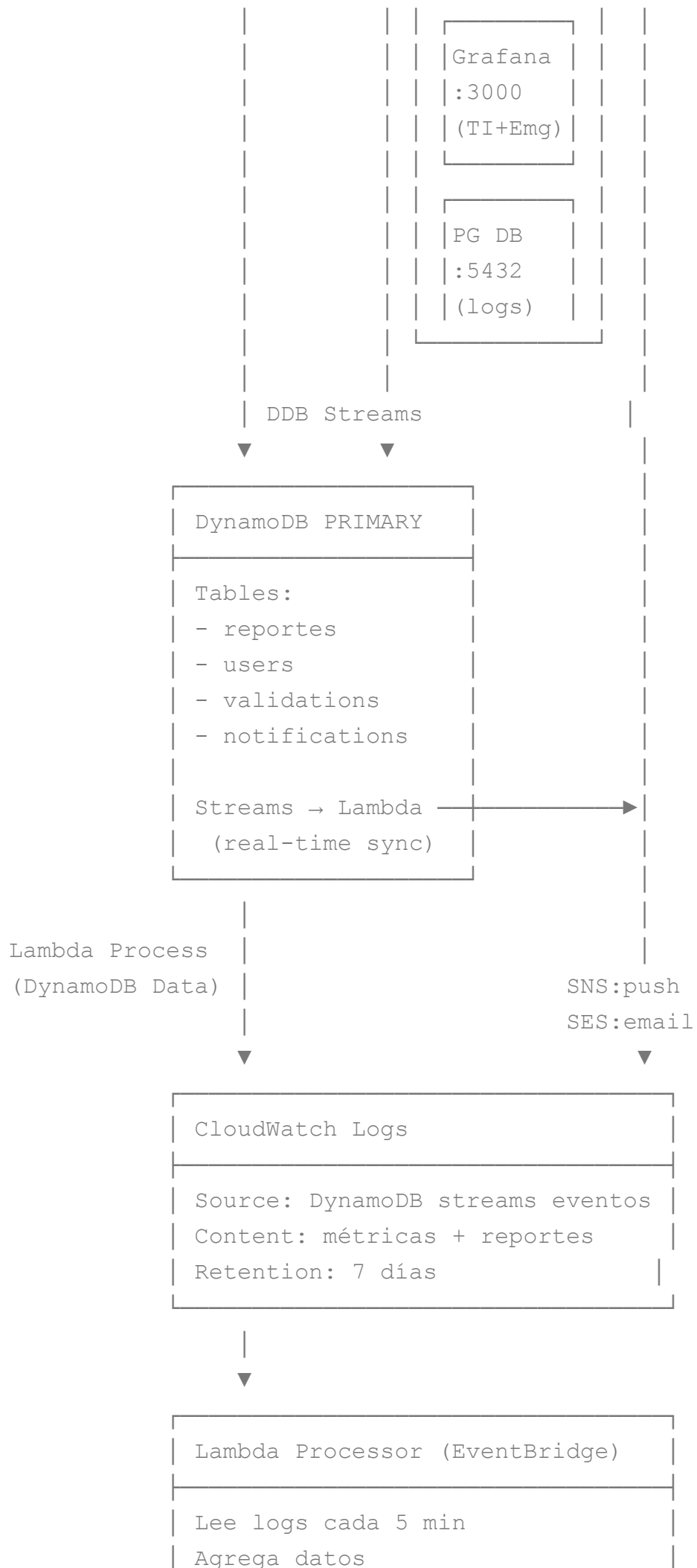
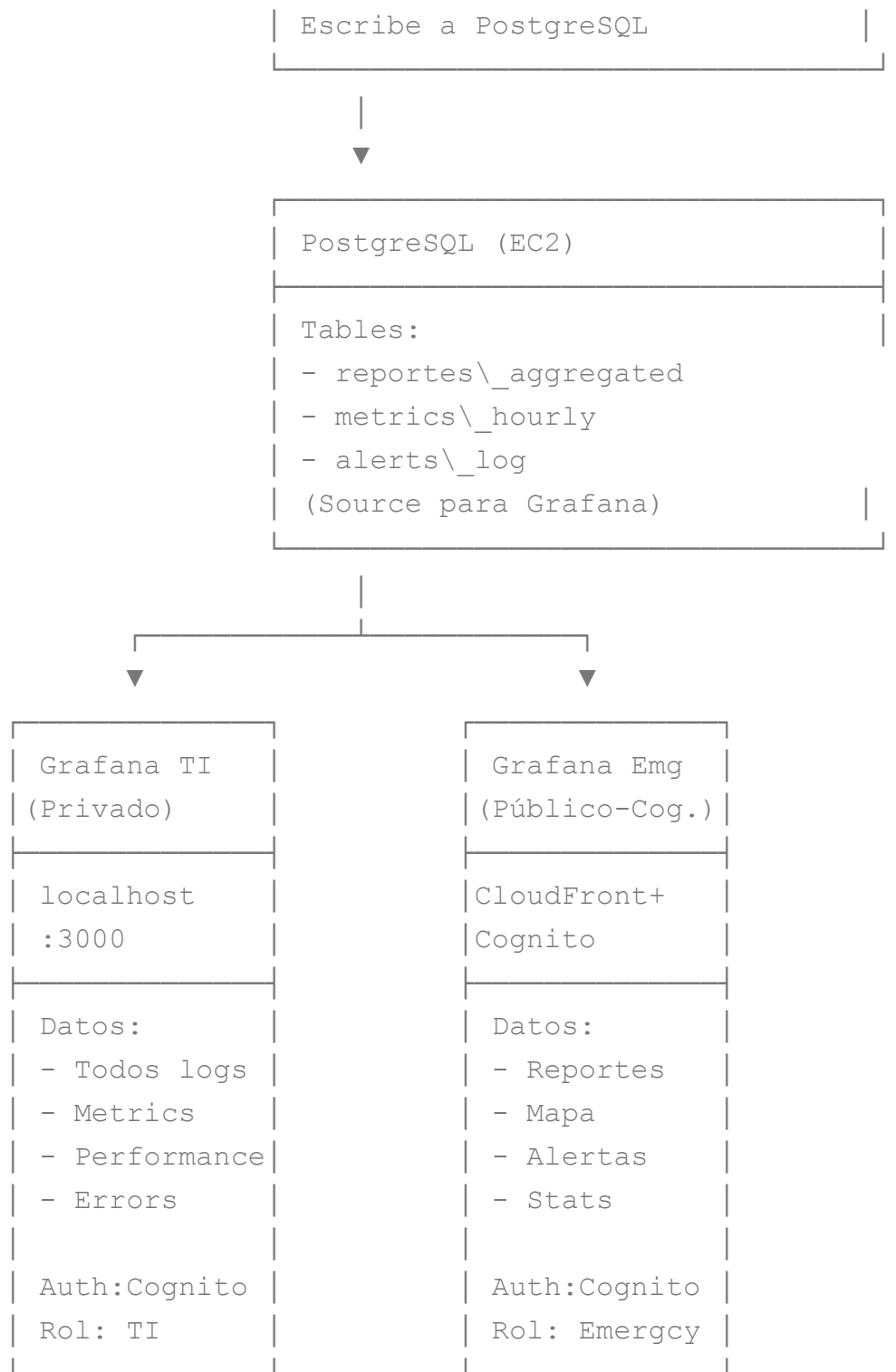


DOCUMENTO 2: ARQUITECTURA DEFENSIBLE

ARQUITECTURA VISUAL







ALMACENAMIENTO:

- └ S3: Imágenes (pre-signed URLs, private)
- └ DynamoDB: Primary data (real-time)
- └ PostgreSQL: Analytics (logs, agregados)



FLUJOS PRINCIPALES

FLUJO 1: Reporte de Incendio (Happy Path)

PASO 1: Captura en Frontend (P1)

- └ Usuario toma foto (cámara)
- └ Ingresa ubicación + detalles
- └ Click "Enviar"
- └ Service Worker intercepta

PASO 2: Offline/Online (P1 + P3)

- └ SI online:
 - └ Envía inmediatamente a API Gateway
- └ SI offline:
 - └ Guarda en IndexedDB
 - └ Muestra: "Sincronizado cuando conexión"
 - └ Service Worker espera evento "online"
- └ JWT en IndexedDB (Cognito)

PASO 3: Lambda Incidencias (P2)

- └ API Gateway → Lambda (autorizado por Cognito)
- └ Recibe: { report_id, location, s3_image_url, timestamp }
- └ Valida estructura (Pydantic)
- └ DynamoDB put_item (tabla reportes)
- └ Dispara evento: DynamoDB Streams

PASO 4: DynamoDB Streams (Serverless)

- └ Evento: INSERT en tabla reportes
- └ Trigger automático → Lambda Notificaciones
- └ Trigger automático → CloudWatch Logs

PASO 5: Lambda Notificaciones (P2)

- └ Recibe evento del stream
- └ Query DynamoDB: usuarios en radio 5km
- └ SNS publish (push notification)
- └ SES send (email alert)
- └ DynamoDB: log notificación enviada

RESULTADO:

- └ DynamoDB: reporte guardado
- └ CloudWatch Logs: evento registrado
- └ SNS: push enviado a 50 usuarios
- └ SES: email enviado a coordinador
- └ Grafana (TI): ve evento en logs (real-time)

FLUJO 2: Validación con IA (Rekognition)

PASO 1: Envío desde Frontend (P1)

- └ S3 pre-signed URL (15 min TTL)
- └ Upload imagen directamente a S3
- └ Recibe: s3://bucket/image_uuid.jpg
- └ Envía URL a Lambda Incidencias

PASO 2: Lambda Incidencias → EC2 (P2)

- └ API Gateway recibe request
- └ Detecta: /validate → rutea a EC2
- └ Payload: { s3_image_url, report_id }

PASO 3: EC2 FastAPI Incendios (P2 + P3)

- └ Recibe s3_image_url
- └ Llama Rekognition API
- └ Rekognition: analiza imagen de S3 (directo, no descarga)
- └ Retorna: { is_fire: true/false, confidence: 0.95, labels: \[...\] }
- └ DynamoDB: almacena resultado validación

PASO 4: Resultado en DynamoDB (P2)

- └ Update table validations
- └ Status: VALIDATED_FIRE | VALIDATED_NO_FIRE | REJECTED
- └ Timestamp: cuando se validó

PASO 5: Synchronous Response (P2)

- └ EC2 → API Gateway → Frontend
- └ Frontend actualiza UI
- └ Muestra: "Validado: Incendio detectado ✓"
- └ Si es falso: "No es incendio, enviado para revisión manual"

COSTO:

- └ Rekognition: \$0 (5k/mes free tier)
- └ S3: \$0 (5GB free, imágenes \~100MB total)
- └ EC2: \$0 (750h free, todo el mes)

FLUJO 3: Autenticación (Cognito)

PASO 1: Signup (Frontend P1)

- └ Formulario: email, password
- └ Cognito valida complejidad

- └ Email: confirmation link

PASO 2: Confirm Email (Frontend P1)

- └ Usuario hace click en link
- └ Cognito: marca email como verified

PASO 3: Login (Frontend P1)

- └ Email + password → Cognito
- └ Cognito retorna: { access_token, refresh_token, id_token }
- └ Frontend: guarda en IndexedDB
- └ JWT válido por 15 min

PASO 4: API Calls (Frontend P1 → Lambda P2)

- └ Cada request incluye: Authorization: Bearer <token>
- └ API Gateway: Cognito Authorizer valida JWT
- └ Cognito OK → Lambda ejecuta
- └ Cognito FAIL → 401 Unauthorized

PASO 5: Refresh Token (Frontend P1)

- └ Si JWT expira:
 - └ Usa refresh_token para obtener nuevo JWT
- └ API Gateway lo permite (sin re-login)
- └ Offline: mantiene último JWT válido + queue

COSTO:

- └ Cognito: \$0 (50k MAU free, equipo = 3 usuarios)
- └ JWT: \$0 (validación local via public key)

COMUNICACIÓN INTER-SERVICIOS

EC2 ↔ Lambda (via API Gateway)

FLUJO:

Lambda Incidencias → API Gateway

- └ POST /validate
 - └ Headers: Authorization: Bearer <JWT>
 - └ Body: { s3_image_url, report_id }
 - └ Route: API Gateway → EC2:8000/validate
 - └ EC2 FASTapi recibe
 - └ Responde JSON

VENTAJAS:

- ✓ Centralizado (API Gateway es punto único)
- ✓ Autenticado (JWT validado)
- ✓ Monitoreado (CloudWatch logs automático)
- ✓ Rate limited (1000 req/min)

LATENCIA:

- └ API Gateway routing: \~50ms
- └ EC2 processing: \~200ms (Rekognition)
- └ Total: \~300ms (acceptable)

DynamoDB → PostgreSQL (via Streams + Lambda)

TRIGGER: DynamoDB Streams

- └ Cada INSERT/UPDATE en DynamoDB
- └ Automáticamente dispara Lambda Processor

LAMBDA PROCESSOR:

- └ Lee evento del stream
- └ Agrega datos (suma, promedio, etc)
- └ Escribe a PostgreSQL
- └ Error: intenta reintentar 3x

VENTAJAS:

- ✓ Real-time (< 1 segundo delay)
- ✓ Asíncrono (no bloquea reportes)
- ✓ Escalable (Streams maneja 100+ req/sec)

COSTO:

- └ DynamoDB Streams: \$0 (incluido)
- └ Lambda: \$0 (1M free requests)
- └ Total: \$0

--



DECISIONES ARQUITECTÓNICAS (ADRs)

ADR-1: API Gateway como punto central

Decisión: Sí, todo pasa por API Gateway

Razón:

- ✓ Cognito Authorizer (auth centralizado)
- ✓ Rate limiting (1000 req/min)
- ✓ CloudWatch logs automático
- ✓ Oculta estructura interna

Alternativa rechazada: Direct Lambda/EC2 calls (sin punto único)

ADR-2: DynamoDB Streams para real-time

Decisión: Sí, usar Streams + Lambda

Razón:

- ✓ Real-time notifications (< 1s)
- ✓ Sin polling (más eficiente)
- ✓ \$0 costo

Alternativa rechazada: Cron job (lag de horas)

ADR-3: PostgreSQL solo para Grafana

Decisión: Sí, lectura desde logs (no direct reads)

Razón:

- ✓ No compite con reportes reales
- ✓ Escalable a 1000+ eventos/sec
- ✓ Datos agregados (no raw)

Alternativa rechazada: Grafana query DynamoDB directo (congestion)

ADR-4: 2 perfiles Grafana

Decisión: Sí, TI (privada) + Emergencias (pública)

Razón:

- ✓ TI ve todo (debugging, performance)
- ✓ Emergencias ve lo importante (mapa, alertas)
- ✓ Ambas con Cognito (seguridad)
- ✓ \$0 costo (mismo Cognito)

ADR-5: IndexedDB para offline

Decisión: Sí, IndexedDB (no localStorage)

Razón:

- ✓ Más seguro (no XSS vulnerable)
- ✓ Mayor capacidad (MB, no KB)
- ✓ Mejor para JSON

Alternativa rechazada: localStorage (limitaciones)

--



VALIDACIÓN ARQUITECTÓNICA

ESCALABILIDAD:

- ✓ 100 reportes/sec: Lambda escala automático
- ✓ 1000 usuarios notificados: SNS maneja
- ✓ Almacenamiento: DynamoDB on-demand

SEGURIDAD:

- ✓ JWT Cognito (estándar)
- ✓ HTTPS TLS (CloudFront + API Gateway)
- ✓ Pre-signed URLs (S3)
- ✓ Security Groups (EC2)

COSTO:

- ✓ Free tier permanente (Lambda, DynamoDB, S3, Cognito)
- ✓ EC2: \$0 (750h/mes)
- ✓ NAT Gateway: ~\$2 en 10 semanas

APRENDIZAJE:

- ✓ Microservicios reales (4 servicios)
- ✓ Event-driven (Streams)
- ✓ Serverless (Lambda)
- ✓ Híbrida (EC2 + Lambda)
- ✓ CI/CD (GitHub Actions)